

Appunti AI - ML - Deep Learning

Alessio Marini

Cose viste da playlist carine su YT

September 01, 2026

1. How do AI and Neural Networks work?

L'**IA** è quel campo che si occupa di creare macchine e sistemi in grado di compiere tasks che normalmente richiederebbero l'intelligenza umana. Il **Machine Learning** è un sottoinsieme dell'AI dove i *modelli* vengono addestrati su dei dati, questi modelli potranno poi essere utilizzati per riconoscere pattern e fare quindi delle predizioni.

Uno dei metodi principali del machine learning sono le **Neural Networks**, queste sono alla base dei famosi *Large Language Model* come ChatGPT.

Una rete neurale è composta da 3 elementi principali, **input nodes**, **hidden layer**, **output nodes**. Se gli hidden layer sono più di 1 allora la rete è detta *deep neural network*. I nodi di una rete prendono il nome di **neuroni**.

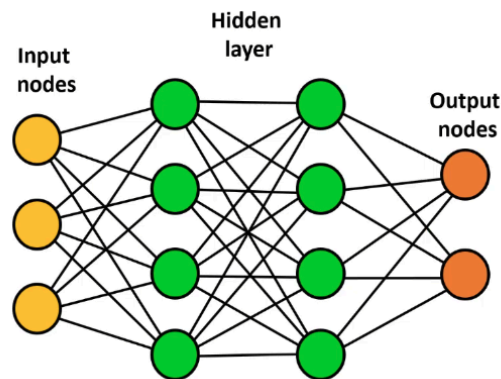


Figura 1: Struttura base di una Neural Network

Per capire più a fondo il funzionamento base, prendiamo delle piccole immagini di $5 \cdot 4 = 20$ pixels, ognuna che rappresenta un numero da 0 a 9.

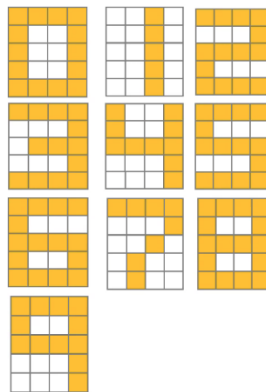


Figura 2: Tutte le immagini

1	1	1	1
1	0	0	1
1	0	0	1
1	0	0	1
1	1	1	1

Figura 3: Ogni pixel è rappresentato da un valore

In questo caso l'1 rappresenta il colore giallo mentre lo 0 rappresenta il bianco.

Possiamo costruire una rete molto semplice composta soltanto da 20 input nodes, uno per ogni pixel, nessun hidden layers e un solo output node. Nell'output node è contenuta la probabilità che il numero sia pari, questa avrà quindi un valore in $[0, 1]$ quindi se il valore è > 0.5 allora il numero è pari mentre se ≤ 0.5 allora il numero è dispari.

Per addestrare alla rete, diremo a questa che le immagini dei numeri pari contengono appunto un numero pari e quindi il loro valore target da raggiungere è 1 mentre per le altre è 0.

Supervised Learning

Questo metodo di apprendimento dove specifichiamo i valori target prima che la rete inizi ad apprendere viene chiamato **Supervised Learning**. Appunto perché possiamo *supervisionare* la rete durante l'apprendimento dato che conosciamo le risposte corrette.

L'addestramento di una rete di solito inizia impostando *casualmente* i pesi della rete, ogni peso determina come un valore di input modifica l'output.

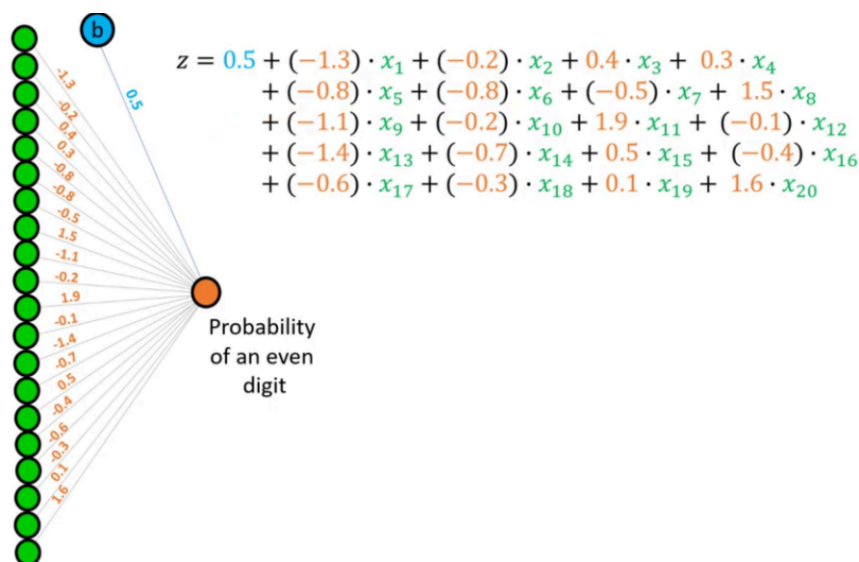


Figura 4: Equazione con pesi, valori in input e bias

In arancione sono appunto indicati i pesi per ciascun input, indicati in verde, infine in blu è indicato un altro valore chiamato *bias*.

Per eseguire l'addestramento quindi vengono inseriti i pixel, ad esempio dell'immagine di 0, in ogni nodo di input, questi valgono o 0 o 1, viene calcolato il risultato finale. Questo risultato non rappresenta una probabilità, per farlo abbiamo bisogno di una **funzione di attivazione** come ad esempio la *Sigmoid Activation Function*:

$$f(z) = \frac{1}{1 + e^{-z}}$$

Dove z indica il risultato dell'equazione precedente. Questa funzione converte i valori in valori all'interno di $[0, 1]$.

Se la rete ottiene un valore corretto allora procede alla prossima immagine, se ottiene un valore sbagliato allora effettua la **Backpropagation** ovvero va a modificare i pesi per fare in modo che la stessa operazione porti ad un risultato corretto.

Questo processo viene appunto chiamato **training** perché durante questo la rete può correggere i suoi pesi e ridurre l'errore. Anche il valore di bias viene aggiustato. Questi vengono modificati da una **loss function** che vedremo più avanti.

L'addestramento di una rete, tramite la ricerca del valore minimo della funzione che rappresenta l'errore, ci permette di trovare il valore ottimale di pesi e bias per avere appunto l'errore minimo, questa funzione potrebbe avere più minimi locali e per questo è importante ripetere lo step di addestramento con valori iniziali diversi per trovare quelli ottimali, ovvero il minimo globale, anche se le reti riescono comunque a performare bene

senza il valore minimo assoluto. Ovviamente una rete così semplice non sarà in grado di “generalizzare” dati più complessi, è per questo che sono importanti gli *hidden layer* che permettono alla rete di rappresentare funzioni più complesse in grado di generalizzare meglio i dati in input.

1.1. Multinomial Logistic Regression

Per prima cosa dividiamo i concetti di *one* e *all* per la logistic regression. Immaginiamo di avere i nostri dati su dei pazienti separati su due colonne:

Status	CRP
No infection	3.8
No infection	5.6
No infection	7.8
No infection	9.2
No infection	14.5
Viral infection	12.9
Viral infection	34.6
Viral infection	30.3
Viral infection	47.0
Viral infection	58.1
Bacterial infection	54.8
Bacterial infection	74.7
Bacterial infection	82.9
Bacterial infection	92.0
Bacterial infection	98.3

Figura 5: Dati dei pazienti

Prendiamo la primo colonna, status, utilizzando la logistic regression binaria dobbiamo creare 3 modelli, in ognuno di questi avremo che ad esempio, i pazienti *no infection* rappresentano 1 e tutti gli altri 0, poi un altro modello dove *viral infection* rappresenta 1 e tutti gli altri 0 e infine *bacterial infection* rappresenta 1 e tutti gli altri 0. Possiamo quindi eseguire questi 3 modelli e vedere poi chi ci restituisce la probabilità più alta.

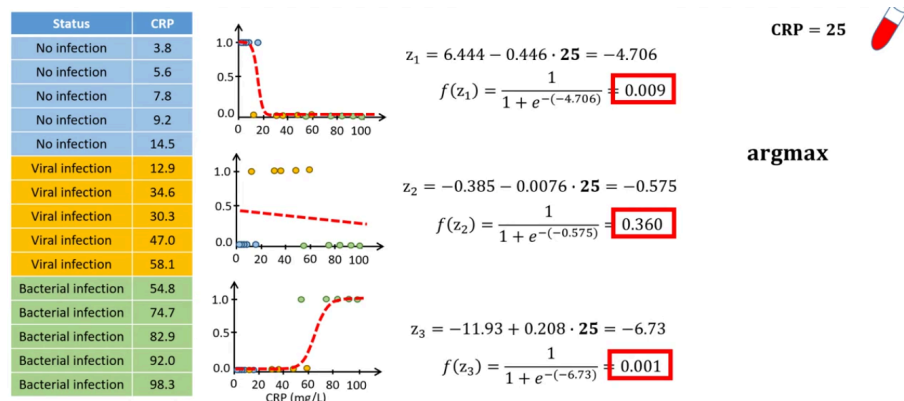


Figura 6: Dati dei pazienti

Notiamo però che i risultati, che sono delle probabilità, non si sommano ad 1. Per fare questo possiamo usare invece la funzione *softmax*:

$$s(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Possiamo vedere ora la **Multinomial Logistic Regression**, possiamo vederla come una stima di $K - 1$ binary logistic regression models dove in questo caso $K = 3$ perché abbiamo 3 classi di output. Per creare soltanto due modelli dobbiamo impostarne uno

come *baseline model* e per farlo sottriamo dai pesi degli altri modelli i pesi del baseline, in questo modo quindi i pesi dei nostri modelli diventano la differenza fra con il baseline.

ANN vs Regression

Il problema principale della Linear Regression è che non tutti i dati sono rappresentabili da una funzione lineare ovvero una linea retta che riesce a *fittarli*. Si potrebbe utilizzare una funzione non lineare ma quando si hanno diversi input potrebbe essere complicato trovare una funzione non lineare adatta. É qui che ci aiutano le neural networks.

Per poter utilizzare le reti neurali di solito si effettua la normalizzazione dei dati, ad esempio con la funzione:

$$x_{\text{norm}} = \frac{x - \min(x)}{\max(x) - \min(x)}$$

In questo caso li portiamo in un range $[0, 1]$. Quando poi otteniamo i risultati dalla rete dovremo de-normalizzarli:

$$x = x_{\text{norm}}(\max(x) - \min(x)) + \min(x)$$

2. Stochastic Gradient Descent

Prendiamo una semplice funzione $y = 2x^2 + 4x + 5$. Possiamo calcolare la derivata ovvero $\frac{dy}{dx} = 4x + 4$ e calcolare quindi la retta adiacente in ogni punto. Possiamo inoltre minimizzare la funzione impostando il risultato a 0 ovvero

$$0 = 4x + 4; x = -1$$

In questo modo sappiamo che quando $x = -1$ la funzione originale sarà al suo valore minimo. Impostando quindi $x = -1$ nella funzione otteniamo $y = 3$ e quindi nella funzione originale non ci sarà nessun altro valore di x tale che $y < 3$.

In machine learning la derivata corrisponde alla loss function, se troviamo il minimo di questa funzione abbiamo trovato dei buoni parametri per fittare i dati.

Per applicare il gradient descent utilizziamo questa formula:

$$x_{\text{new}} = x_{\text{old}} - \gamma \nabla f(x_{\text{old}})$$

Dove inizialmente avremo x_{old} inizializzato ad un valore casuale mentre γ prende il nome di *learning rate*. Facendo quindi la differenza fra questo valore casuale e il valore della derivata in quel punto moltiplicato per il learning rate otteniamo il nuovo punto verso il minimo della derivata. È importante impostare un learning rate corretto, ad esempio uno troppo alto ci porterebbe a rimbalzare fra i punti della funzione senza mai raggiungere il minimo.

Il metodo contiene la parola *stochastic* dato che per effettuare questi calcoli vengono presi dati in modo casuale dal set di addestramento, in questo modo non è sempre detto che il valore scenda verso il minimo ma potrebbe anche tornare indietro, questo però è utile anche per «scappare» da minimi locali.

2.1. Mini-batch gradient descent

Funziona come il metodo classico ma i dati vengono presi in sottoinsiemi e l'aggiornamento dei pesi avviene alla fine dell'iterazione di ogni mini-batch. Questo porta ad un risparmio di memoria ma anche a risultati diversi da mini-batch a mini-batch. Quando abbiamo iterato su tutte le batch abbiamo compiuto un'*epoch*.

Gradient

Tutto quello che abbiamo visto finora lo abbiamo fatto pensando di aggiornare un solo parametro alla volta per calcolare la derivata, quindi fissando ad un valore costante gli altri. Nella realtà il calcolo della derivata avviene per tutti i parametri che dobbiamo ottimizzare e questo prende appunto il nome di gradiente ovvero la derivata calcolata su ogni parametro impostando ad una costante tutti gli altri.

3. Deep Learning

Un problema principale del deep learning è l'overfitting, ovvero quando la rete “impara” troppo bene dai dati di addestramento e questo la porta a generalizzare male su dati nuovi. I principali metodi per mitigare questo fenomeno sono l'**Early Stopping** e il **dropout**.

Per l'early stopping abbiamo bisogno dei dati di training e di validation. Abbiamo bisogno di guardare l'errore di validation durante quest'ultima, infatti quello di training scenderà praticamente sempre ma quello di validation potrebbe iniziare a risalire se la rete va in overfitting, è in questo momento che dobbiamo interrompere l'addestramento perché la rete ha raggiunto dei pesi ottimali per la generalizzazione.

Il dropout invece avviene durante il training, questo spegne in modo randomico dei neuroni durante l'addestramento, questo forza la rete a non fare troppo affidamento a neuroni specifici ma ad imparare veramente i pattern e non i dati a memoria.

4. Convolutional Neural Network

Rappresentiamo le immagini in modo che abbiano $5 \cdot 5 = 25$ pixels. Vediamo meglio, ad esempio, l'immagine che rappresenta lo zero.

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Figura 7: Immagine che rappresenta la cifra 0

Nelle CNN si utilizzano i *filter*, anche chiamati *kernel*. In questo esempio utilizziamo un kernel 2×2 ovvero una matrice composta da:

$$\begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$$

Questi valori sono stati ottimizzati tramite il training per questo esempio. Il filtro viene passato sopra l'immagine, pixel per pixel, e calcoliamo un *dot product* ovvero moltiplichiamo i valori che si sovrappongono poi li sommiamo e inseriamo il risultato nella *feature map*.

Stride

Il numero di pixel di quanto muoviamo il filtro si chiama *stride*. In questo caso muovendo il filtro su ogni pixel abbiamo $\text{stride} = 1$.

Quindi con l'immagine dello zero otteniamo:

Image 5 x 5					Feature map			
1	1	1	1	0	1	1	3	1
1	0	0	1	0	1	0	2	1
1	0	0	1	0	1	0	2	1
1	0	0	1	0	1	0	2	1
1	1	1	1	0	3	2	2	1

Questa operazione prende il nome di **Convolution** e serve appunto a trovare delle feature locali nell'immagine. Una volta ottenuta la feature map applichiamo una funzione di attivazione come ad esempio la *ReLU*:

$$f(x) = \max(0, x)$$

La ReLU, in breve, imposta a 0 tutti i valori negativi e lascia invariati quelli positivi.

Una volta fatte queste operazioni effettuiamo il *pooling*, ovvero passiamo un altro filtro e prendiamo il valore massimo contenuto in quel filtro, questo valore viene inserito in una nuova griglia chiamata **pooled feature map**. Infine schiacciamo questa griglia in un array monodimensionale. Ottenendo come pipeline principale:

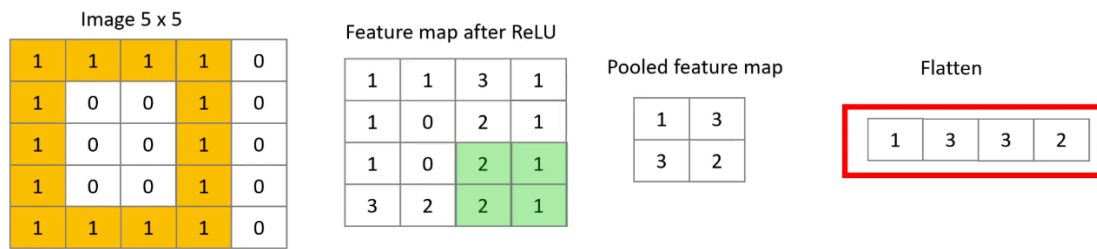
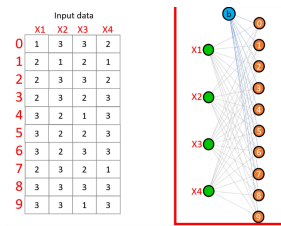


Figura 9: Intero processo per immagine zero

Andiamo ad effettuare queste operazioni per tutte le immagini dei numeri da 0 a 9 e creiamo una semplice rete neurale, per semplicità senza hidden layer:



Dato che le pooled feature map hanno soltanto 4 feature per le immagini la nostra rete avrà solo 4 nodi in input, ricordiamo che ad esempio una rete classica senza convolution ne avrebbe richiesti 25, uno per ogni pixel. Inoltre dato che dobbiamo predire 10 valori avremo 10 nodi in output.

Spesso lavorando con le CNN si utilizzano più filtri. Nel nostro caso ad esempio utilizzandone 2 avremo ottenuto 2 matrici fino ad arrivare a 2 array e quindi la rete avrebbe richiesto 8 nodi in input.

Anche i filtri possono essere ottimizzati durante l'addestramento, vengono inizializzati con pesi casuali e durante l'addestramento vengono modificati, se facciamo un calcolo con questa nuova configurazione del doppio filtro avremo in totale 8 valori da addestrare per i 2 filtri, 4 ciascuno; poi abbiamo 8 pesi per ciascun nodo in input collegato ai 10 nodi di output quindi $8 \cdot 10 = 80$ pesi e infine i 10 bias uno per ogni nodo output. In totale otteniamo $8 + 10 + 80 = 98$ pesi da addestrare.

4.1. Immagini a colori

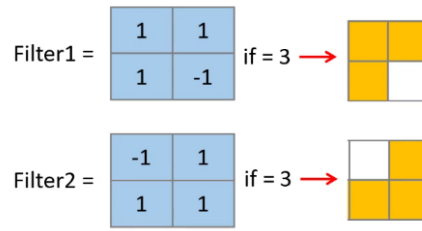
Per quanto riguarda le immagini a colori queste sono composte non più da una sola matrice ma da 3, una per ogni canale ovvero *rosso*, *verde* e *blu*. Se utilizziamo un filtro, lo applichiamo su tutti i canali e facciamo gli stessi calcoli, poi sommiamo i 3 valori ottenuti e andiamo a comporre la feature map. Da notare che è possibile aggiungere un bias anche in questa fase.

4.2. Come fanno i filtri ad identificare le feature?

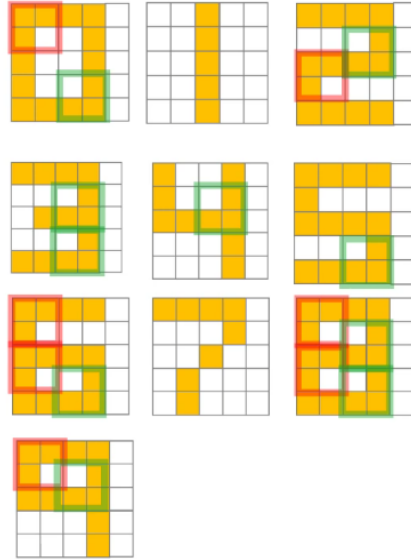
Prendiamo come esempio questi due filtri:

$$\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad \begin{bmatrix} -1 & 1 \\ 1 & 1 \end{bmatrix}$$

Se il risultato del dot product è uguale a 3 allora significa che l'immagine contiene un angolo, rispettivamente per i due filtri abbiamo questi angoli:



Andiamo a vedere dove sono presenti questi angoli nelle immagini:



Come possiamo notare alcune immagini contengono un solo tipo di angolo, oppure altre ancora ne contengono un numero specifico diverso da tutte le altre. In ogni caso la cosa “potente” che differenzia le CNN dalle classiche reti neurali è che possiamo muovere l’immagine come vogliamo ma queste saranno comunque in grado di riconoscere queste feature come ad esempio un angolo, debolezza invece delle classiche reti che cercano di imparare a riconoscere le immagini dalla semplice posizione dei pixel.

É per questo che si utilizzano vari filtri sulla stessa immagine, filtri diversi riescono a identificare feature diverse.

4.3. Padding

Abbiamo visto che applicare layer convoluzionali va a ridurre la feature map di dimensione, se ne usiamo tanti potremmo arrivare ad una dimensione tale da rendere la feature map inutile. Un altro problema inoltre è che i pixel ai bordi dell’immagine hanno meno influenza degli altri, questi infatti vengono utilizzati meno volte all’interno dei filtri.

Un modo per risolvere questi problemi è aggiungere il padding ovvero dei pixel contenenti 0 ai bordi esterni, questo ci permette di utilizzare lo stesso numero di volte i pixel esterni e inoltre ottenere una feature map più grande dell’immagine originale.

Per calcolare in modo corretto l’output della feature map possiamo utilizzare la seguente formula:

$$\left\lceil \frac{n + 2p - f}{s} \right\rceil + 1$$

Dove:

- n : Larghezza (o altezza) dell'immagine in input
- f : Larghezza (o altezza) del kernel
- p : Numero di layer per il padding
- s : Stride ovvero il numero di pixel con cui si muove il kernel

Infatti utilizzando l'esempio precedente con immagine 5×5 e un layer di padding, con il kernel 2×2 con stride 1 abbiamo:

$$\left\lceil \frac{5 + 2 \cdot 1 - 2}{1} \right\rceil + 1 = 6$$

Ovvero un pixel più larga dell'immagine originale.

5. Autoencoders

Gli autoencoders sono delle reti neurali che possono essere addestrate per fare in modo che riproducano, al meglio delle loro capacità, i dati che gli sono stati forniti in input.

Questi sono composti da dei nodi di input, uno o più hidden layer e tanti nodi in output quanti quelli in input. Di solito, gli hidden layer sono composti da meno nodi di quelli in input.

Il primo passaggio dei dati dagli input nodes ai primi hidden layers prende il nome di *encoding* ovvero la compressione dei dati in una dimensione inferiore. Quando i dati escono dagli hidden layer e vanno negli output node allora siamo nella fase di *decoding* infatti tornano alla dimensione originale:

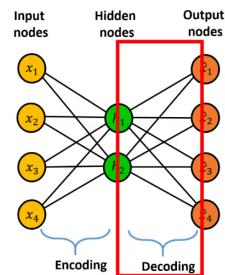


Figura 13: Fasi dell'autoencoder

Ad esempio già dopo aver trainato i pesi dagli input nodes verso gli hidden layer abbiamo un metodo per trasformare 4 feature in 2 e, ad esempio, visualizzarle più facilmente in un grafico e identificare i record che sono più simili fra loro.

6. Transfer Learning

Il transfer learning si basa sul trasferire la «conoscenza» di una rete pre-addestrata ad un'altra rete ovviamente con compiti simili. Ad esempio una rete che riconosce gli animali come lupo, tigre ed elefante potrebbe essere la nostra rete pre-addestrata con migliaia di immagini, mentre la seconda rete vogliamo che riconosca cani e gatti ma abbiamo a disposizione meno foto.

In questo esempio dato che lupo e tigre sono abbastanza simili alle classi cane e gatto possiamo prendere l'intera rete e sostituire semplicemente il layer con i nodi di output. Ovviamente dobbiamo prendere anche gli stessi pesi già addestrati che magari sapranno riconoscere dettagli come orecchie, occhi ecc..., gli unici pesi che cambiano saranno quelli finali che collegano l'ultimo hidden layer ai nodi di output.

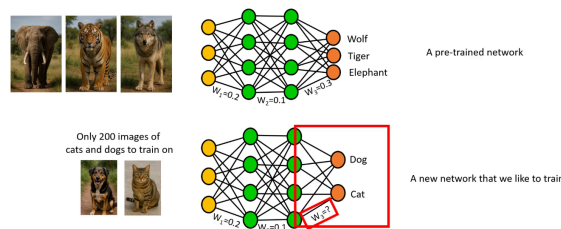


Figura 14: Transfer Learning

Se invece la rete preaddestrata era appunto addestrata su dati molti diversi dal nostro scopo allora avrebbe senso anche «scongellare» i pesi fra i vari hidden layer per permettere alla rete di imparare a riconoscere nuove feature mai viste prima.

Per vedere meglio il funzionamento del transfer learning ci basiamo sulla rete VGG16 trainata sul dataset ImageNet:

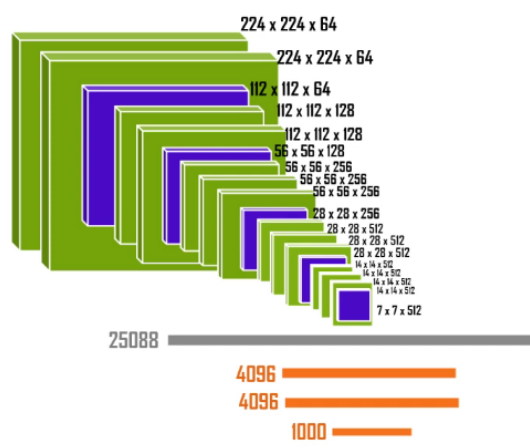


Figura 15: Struttura VGG16

In verde sono indicati i convolutional layer mentre in blu i pooling layer che rendono appunto le matrici di dimensione minore. Nell'ultimo strato della rete si ottengono 512 matrici di dimensione 7×7 che vengono poi appiattite in un array di 25088 elementi che funge da input per i fully connected layer, l'output layer ha 1000 nodi dato che il dataset ha 1000 categorie di elementi che riesce a classificare.

La rete utilizza dei filtri su 3 livelli, ovvero quelli dei colori.

Ad esempio i primi layer potrebbero essere addestrati per identificare features come angoli e segmenti mentre quelli più profondi elementi come naso, orecchie e zampe mentre quelli finali riescono a identificare cose più complesse come la testa.

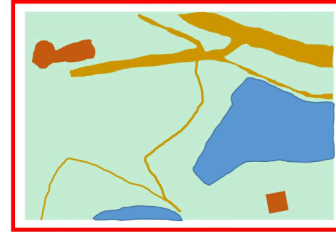
Per applicare il transfer learning potremmo ad esempio mantenere tutta la parte con i layer convoluzionale ed eliminare i fully connected layer per sostituirli con dei layer più semplici in grado di identificare soltanto cani e gatti. In questo modo otteniamo una rete con milioni di parametri già addestrati e che non dobbiamo modificare e una parte, con molti meno pesi da addestrare.

7. U-Net

La U-Net è stata sviluppata principalmente per la segmentazione nelle immagini. Prendiamo ad esempio un'immagine satellitare e immaginiamo di voler creare una segmented map, anche chiamata *mask*:

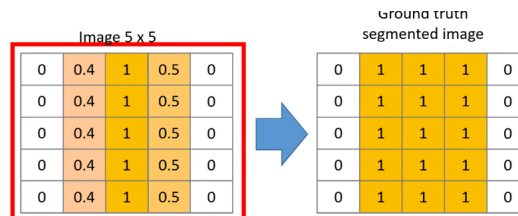


Satellite image



A segmented image (mask)

Per vedere come funzionano, prendiamo una semplice immagine 5×5 e immaginiamo di voler creare la sua segmented image:

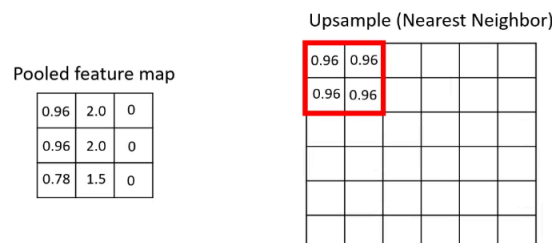


L'immagine è stata normalizzata in modo che il valore di ogni pixel è rappresentato da un numero, più è vicino ad 1 più il pixel è luminoso mentre lo 0 rappresenta il background. Dato che i pixel centrali non fanno parte del background diremo che fanno parte del *foreground*.

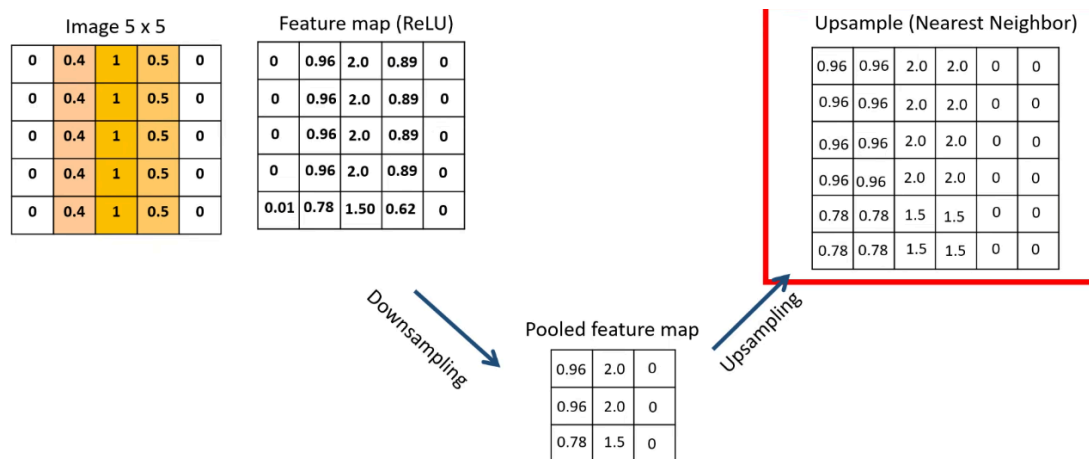
La mask sarà composta da pixel di valore 1 e 0 dove gli 1 rappresentano il foreground ovvero l'elemento da segmentare.

Il primo passo all'interno di una U-Net è quello di creare una feature map che estrae appunto features locali dall'immagine, andrà quindi usato un filtro da ottimizzare nel training. Dopo la feature map applichiamo una activation function come la ReLU.

Successivamente viene fatto pooling ovvero prendere il valore massimo in una finestra che passa sopra la feature map. Adesso che siamo passati ad una matrice molto più piccola rispetto a quella iniziale applichiamo l'*upsample* con il metodo del *nearest neighbor* ovvero prendiamo il primo elemento della pooled feature map e lo copiamo in tutti i suoi vicini:

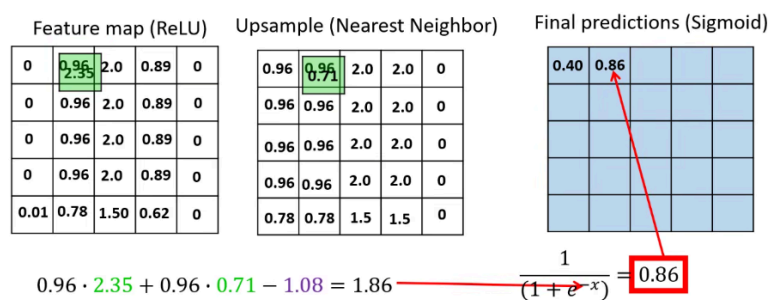


Quindi come pipeline totale abbiamo:



Questa forma appunto ad “U” dà il nome alla rete.

Un altro step della U-Net è quello dalle *Skip Connection* ovvero si prende la Feature Map prima del downsampling e si concatena a quella ottenuta dal upsampling, tagliando via i pixel di quella più grande per farle combaciare in dimensione. Calcoliamo poi un’ultima convoluzione con un filtro $1 \times 1 \times 2$ ovvero lo facciamo scorrere su ogni pixel di entrambe le matrici, il risultato lo inseriamo una activation come la sigmoid e lo inseriamo nella matrice risultante finale:



Filter = Kernel ($1 \times 1 \times 2$)

Questi valori corrispondono alla probabilità se quei pixel nell’immagine fornita in input corrispondono al foreground o al background.

7.1. Original U-Net (2015)

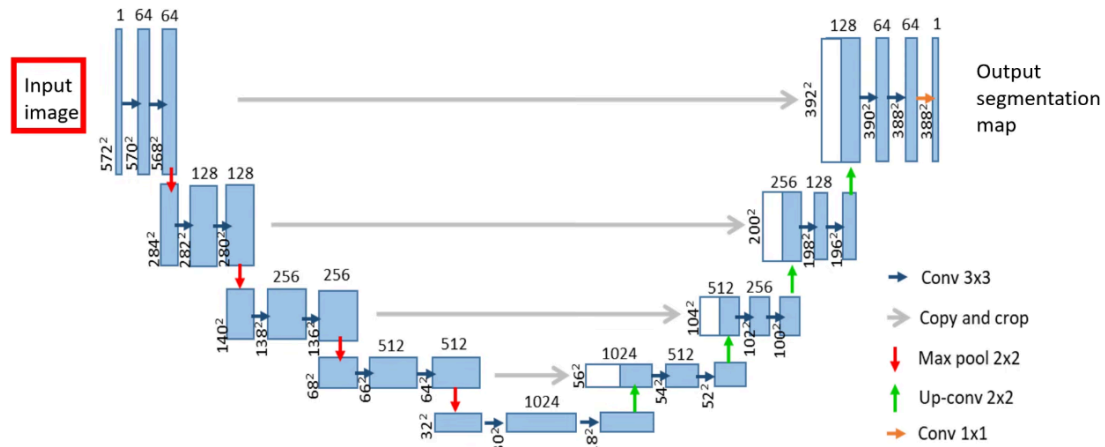


Figura 21: Pipeline della U-Net (2015)

Come input abbiamo un'immagine in scala di grigi, per questo la rete inizia con un solo canale ovvero una matrice 2D, l'immagine viene riscalata in 572×572 . Per il primo step di convolution utilizzano 64 filtri diversi di grandezza 3×3 , infatti come secondo layer abbiamo 64 feature map, ogni convolution è seguita da una ReLU. Viene fatta un'altra convolution sulle 64 feature maps utilizzando filtri 3×3 dato che deve essere applicato uno su ciascuna feature map.

Viene fatto il max-pooling con un kernel 2×2 con stride 2 e per ogni feature map si utilizzano due filtri arrivando a 128 feature map. Si continua fino ad arrivare a delle feature map 28×28 e si utilizza la transposed convolution con 512 filtri per l'upsampling.

Tutte le feature map upscalate sono affiancate con quelle precedenti con le skip connection.

La Skip Connection serve a "riprendere" le informazioni spaziali perse durante le fasi di downsampling per via del pooling.

La parte sinistra della rete che effettua il downsampling si chiama **Encoder** e serve a individuare le features nell'immagine. La parte destra che effettua l'upsampling prende il nome di **Decoder** e serve a localizzare dove le features sono localizzate nell'immagine.

7.2. Evaluation

Per valutare le performance della rete possiamo utilizzare diverse misure, la più semplice è la *pixel accuracy*:

$$\text{Pixel Accuracy} = \frac{\text{Number of correctly predicted pixels}}{\text{Total number of pixels}}$$

$$\text{Pixel Error} = 1 - \text{Pixel Accuracy}$$

La pixel accuracy però potrebbe portarci fuori strada, ad esempio se l'immagine ha come foreground una zona piccolissima e la nostra rete predicta tutto come background avrà comunque una pixel accuracy molto alta anche se effettivamente non ha individuato nulla.

Una misura migliore è l'**IoU (Intersection over Union)**:

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} = \frac{\text{Intersection}}{\text{Union}}$$

Un'altra è la **Dice**:

$$\text{Dice} = \frac{2 \cdot |A \cap B|}{|A| + |B|}$$